

MOST ASKED DSA INTERVIEW QUESTIONS

(0-3 YEARS) (5-20 LPA)

PAGE 1

1. ARRAYS – MOST ASKED QUESTIONS



Arrays are the foundation of DSA. Most of the interview questions for 0-3 YOE are based on Arrays.

#	QUESTION	PATTERN	EXPECTED COMPLEXITY	KEY IDEA / HINT
①	Two Sum (LeetCode 1)	Hashing	$O(n)$ time $O(n)$ space	- Use a HashMap to store number $\rightarrow$ index - For each number, check if target - num exists
②	Best Time to Buy & Sell Stock (LC 121)	One Pass / Greedy	$O(n)$ time $O(1)$ space	- Keep track of minimum price so far - Max profit = max(sell - min_buy)
③	Maximum Subarray (Kadane's Algo) (LC 53)	Kadane's Algorithm	$O(n)$ time $O(1)$ space	- Keep running sum - If sum < 0, reset to 0 - Update max_sum
④	Move Zeroes (LC 283)	Two Pointers	$O(n)$ time $O(1)$ space	- Place all non-zero elements at front - Fill remaining with 0
⑤	Majority Element (LC 169)	Boyer-Moore Voting Algo	$O(n)$ time $O(1)$ space	- Count / cancel pairs - Candidate will be the majority
⑥	Merge Sorted Arrays (LC 88)	Two Pointers	$O(n + m)$ time $O(1)$ space	- Start from the end - Place larger element at the end of nums1
⑦	Product of Array Except Self (LC 238)	Prefix & Suffix	$O(n)$ time $O(1)$ space (ignoring output)	- Left product & Right product - res[i] = left[i] * right[i]



IMPORTANT PATTERNS TO RECOGNIZE

- $\rightarrow$ Hashing Pattern
- $\rightarrow$ Subarray / Kadane's Pattern
- $\rightarrow$ Two Pointers Pattern
- $\rightarrow$ Prefix / Suffix Pattern
- $\rightarrow$ Sliding Window Pattern
- $\rightarrow$ Sorting / Greedy Pattern



TIP:

Understand the pattern behind the question, not just the solution.



GOAL:

Recognize pattern in < 30 sec & choose the right approach.

MOST ASKED DSA INTERVIEW QUESTIONS

(0-3 YEARS) (5-20 LPA)

PAGE 2

2. STRINGS - MOST ASKED QUESTIONS

★ String questions test your basics of characters, hashing, pointers and pattern recognition.

#	QUESTION	PATTERN	EXPECTED COMPLEXITY	KEY IDEA / HINT
①	Valid Palindrome (LC 125)	Two Pointers	$O(n)$ time $O(1)$ space	- Compare from both ends - Skip non-alphanumeric - Case insensitive comparison
②	Valid Anagram (LC 242)	Hashing (Frequency Count)	$O(n)$ time $O(1)$ space	- Count frequency of characters - Two strings are anagrams if all counts match
③	Longest Common Prefix (LC 14)	Horizontal Scan (or Vertical)	$O(S)$ time $O(1)$ space	- Compare characters at same index - Stop when mismatch found (S = total characters)
④	Longest Substring Without Repeating Characters (LC 3)	Sliding Window	$O(n)$ time $O(k)$ space (k = charset size)	- Use HashMap / Set - Expand window, shrink on duplicate - Keep max length
⑤	Group Anagrams (LC 49)	Hashing (Sorted String / Frequency)	$O(n * k \log k)$ time $O(n * k)$ space (k = avg length)	- Sort each string or use frequency map as key - Group strings with same key
⑥	String Compression (LC 443)	Two Pointers	$O(n)$ time $O(1)$ space	- Read & write pointers - Count consecutive chars - Replace count > 1 with number

★ PATTERN RECOGNITION - STRINGS

→ Two Pointers → Palindrome, Reverse, Compare → Prefix Matching → Common Prefix, Search
 → Hashing → Anagram, Frequency, Grouping → Simulation → Compression, Encoding
 → Sliding Window → Longest Substring problems → String Manipulation → Build, Transform, Parse



STRINGS - QUICK TIPS

- ✓ Think of character handling first (case, spaces, special chars).
- ✓ Decide if you need hashing, two pointers or sliding window.
- ✓ For optimization → try to reduce extra space.

TIME COMPLEXITY GUIDE

$O(1)$ → Constant
 $O(\log n)$ → Logarithmic
 $O(n)$ → Linear
 $O(n \log n)$ → Linearithmic
 $O(n^2)$ → Quadratic



Master the patterns, and the questions become easy!
Practice regularly and solve with different approaches.



MOST ASKED DSA INTERVIEW QUESTIONS

(0-3 YEARS) (5-20 LPA)

PAGE 3

3. TWO POINTERS + SLIDING WINDOW - MOST ASKED QUESTIONS

- ★ Two Pointers solve problems on sorted / unsorted arrays efficiently.
Sliding Window is used for subarray / substring problems.

#	QUESTION	PATTERN	EXPECTED COMPLEXITY	KEY IDEA / HINT
①	Container With Most Water (LC 11)	Two Pointers (Opposite Ends)	$O(n)$ time $O(1)$ space	- Move the pointer at smaller height - Area = $\min(h_1, h_2) \times \text{width}$
②	3Sum (LC 15)	Two Pointers + Sorting	$O(n^2)$ time $O(1)$ space (extra for sort)	- Sort the array - Fix one element - Use two pointers for remaining
③	Remove Duplicates from Sorted Array (LC 26)	Two Pointers	$O(n)$ time $O(1)$ space	- Slow pointer for unique position - Fast pointer to scan - Copy when new unique found
④	Longest Substring Without Repeating Characters (LC 3)	Sliding Window + Hashing	$O(n)$ time $O(k)$ space ($k = \text{charset size}$)	- Expand window to include - Shrink window on duplicate - Keep max length
⑤	Maximum Sum Subarray of Size K (LC 643)	Sliding Window	$O(n)$ time $O(1)$ space	- Calculate sum of first window - Slide window, add new, remove old - Track max sum
⑥	Minimum Window Substring (LC 76)	Sliding Window + Hashing	$O(n)$ time $O(\sigma)$ space ($\sigma = \text{unique chars}$)	- Expand to include all chars - Shrink to minimum valid window - Track min length window

★ WHEN TO USE WHAT?

→ Use TWO POINTERS when:

- You need pairs/triplets
- Array is sorted / can be sorted
- Problem asks for min/max with two values

→ Use SLIDING WINDOW when:

- Subarray / substring is involved
- Need longest/shortest with some condition
- Elements are positive / count based

TWO POINTERS TEMPLATE

```

left = 0, right = n-1
while left < right:
    if condition:
        do something
        left += 1 or right -= 1
    else:
        other operation
        left/right move accordingly
    
```



Move pointers carefully and think about how each move affects the answer.

SLIDING WINDOW TEMPLATE

```

left = 0
for right in range(n):
    add nums[right] / expand
    while condition not valid:
        remove nums[left] / shrink
        left += 1
    update answer
    
```



Practice these patterns until they become your instinct.
Pattern recognition + Consistent practice = Success in Interviews!



MOST ASKED DSA INTERVIEW QUESTIONS

(0-3 YEARS) (5-20 LPA)

PAGE 4

4. HASHING + PREFIX SUM - MOST ASKED QUESTIONS

- ★ Hashing helps in fast lookups and frequency based problems.
Prefix Sum helps in subarray / range based problems.

#	QUESTION	PATTERN	EXPECTED COMPLEXITY	KEY IDEA / HINT
①	Two Sum (HashMap) (LC 1)	Hashing (Map)	$O(n)$ time $O(n)$ space	- Store num $\rightarrow$ index in HashMap - For each num, check target - num - Return indices if found
②	Subarray Sum Equals K (LC 560)	Prefix Sum + Hashing	$O(n)$ time $O(n)$ space	- Keep running prefix sum - If (prefix - k) exists in map, then valid subarray found - Map stores prefixSum $\rightarrow$ count
③	Longest Consecutive Sequence (LC 128)	Hashing (Set)	$O(n)$ time $O(n)$ space	- Put all numbers in a Set - Check only if (num-1) not in set - Count consecutive numbers
④	Top K Frequent Elements (LC 347)	Hashing + Heap / Bucket	$O(n \log k)$ or $O(n)$ (bucket sort)	- Count frequency with HashMap - Use Min-Heap of size k or Bucket sort by frequency
⑤	Subarray with Given Sum (K) (LC 325)	Prefix Sum + Hashing	$O(n)$ time $O(n)$ space	- Store prefixSum $\rightarrow$ earliest index - If (prefix - k) exists, update max length
⑥	Count of Nice Subarrays (LC 1248)	Prefix Sum + Hashing	$O(n)$ time $O(n)$ space	- Convert to 0/1 (odd = 1) - prefixSum[i] - prefixSum[j] = k $\Rightarrow$ use map of prefixSum freq

★ $O(n^2)$ TO $O(n)$ OPTIMIZATION USING HASHING / PREFIX SUM

Brute Force ($O(n^2)$)

- Use two loops
- Check all pairs / subarrays
- Causes Time Limit Exceeded (TLE)



Optimized ($O(n)$)

- Use HashMap / Prefix Sum
- Store previously seen info
- Convert nested loops into single loop

PREFIX SUM BASICS

prefix[i] = sum of elements from index 0 to i
Subarray sum from i to j:

$$\text{sum}(i, j) = \text{prefix}[j] - \text{prefix}[i-1]$$

Helps to answer range / subarray queries fast.

HASHING BASICS

Key $\rightarrow$

HashMap / Set	
Key1	$\rightarrow$ Value1
Key2	$\rightarrow$ Value2
...	...
...	...

- Almost $O(1)$ average lookup / insert
- Very useful for counting, checking existence, grouping

COMMON HASHING PATTERNS



Frequency Counting



Two Sum Pattern



Grouping (Anagram)



Prefix Sum Pattern



Graph / Visited Set



Master Hashing & Prefix Sum $\rightarrow$ Solve Faster, Write Better Code!



MOST ASKED DSA INTERVIEW QUESTIONS

(0-3 YEARS) (5-20 LPA)

PAGE 5

5. LINKED LIST - MOST ASKED QUESTIONS

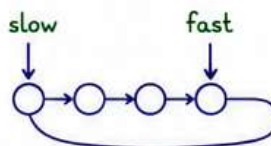
- ★ Linked List questions test pointer manipulation and edge case handling.
Fast & Slow pointer technique is used in many problems.

#	QUESTION	PATTERN	EXPECTED COMPLEXITY	KEY IDEA / HINT
①	Reverse Linked List (LC 206)	Iterative (Three Pointers)	$O(n)$ time $O(1)$ space	- Keep prev, curr, next - Reverse the links one by one - Return prev at the end
②	Linked List Cycle (LC 141)	Fast & Slow Pointers (Floyd's Algo)	$O(n)$ time $O(1)$ space	- Move slow by 1, fast by 2 - If they meet, cycle exists - If fast reaches end, no cycle
③	Middle of Linked List (LC 876)	Fast & Slow Pointers	$O(n)$ time $O(1)$ space	- Move slow by 1, fast by 2 - When fast reaches end, slow will be at middle
④	Merge Two Sorted Lists (LC 21)	Two Pointers	$O(n + m)$ time $O(1)$ space	- Compare nodes of both lists - Attach smaller node to result - Move pointer of that list
⑤	Remove Nth Node From End (LC 19)	Two Pointers (Gap Technique)	$O(n)$ time $O(1)$ space	- Move fast n steps ahead - Move both until fast reaches end - Delete slow.next
⑥	Intersection of Two Linked Lists (LC 160)	Two Pointers (Switch Lists)	$O(n + m)$ time $O(1)$ space	- Traverse both lists - Switch head when you reach end - They will meet at intersection or become NULL

★ FAST & SLOW POINTER - WHEN TO USE?

Use Fast & Slow when:

- ✓ Detecting cycle in Linked List
- ✓ Finding middle element
- ✓ Finding start of cycle
- ✓ Finding k-th node from end



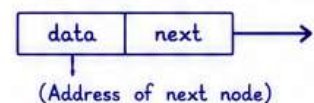
Why it works?

- ✓ Fast moves 2x, slow moves 1x
- ✓ If cycle exists, fast will lap slow
- ✓ If no cycle, fast reaches NULL

COMMON LINKED LIST OPERATIONS COMPLEXITY

- | | | | |
|------------------|----------|----------------------|----------|
| • Traverse | → $O(n)$ | • Insert at Tail | → $O(n)$ |
| • Search | → $O(n)$ | • Delete at Tail | → $O(n)$ |
| • Insert at Head | → $O(1)$ | • Insert at Position | → $O(n)$ |
| • Delete at Head | → $O(1)$ | • Delete at Position | → $O(n)$ |

NODE STRUCTURE



PATTERN RECOGNITION - LINKED LIST

- Fast & Slow Pointers → Cycle / Middle / k-th from end
- Two Pointers → Merge / Intersection
- Dummy Node → Easy handling of edge cases



TIP:

Always handle edge cases first:
Empty list, single node,
two nodes, etc.



Practice these questions until pointer manipulation becomes your strength! 🚀



MOST ASKED DSA INTERVIEW QUESTIONS

(0-3 YEARS) (5-20 LPA)

PAGE 6

6. STACKS + QUEUES – MOST ASKED QUESTIONS



Stack & Queue problems test your understanding of LIFO, FIFO, and monotonic thinking.

#	QUESTION	PATTERN	EXPECTED COMPLEXITY	KEY IDEA / HINT
①	Valid Parentheses (LC 20)	Stack	$O(n)$ time $O(n)$ space	- Push opening brackets - For closing, check top of stack - Stack must be empty at end
②	Min Stack (LC 155)	Stack (Extra Space)	$O(1)$ time for push, pop, top, getMin $O(n)$ space	- Maintain another stack for mins - Or store (val, current_min) - All operations in $O(1)$
③	Next Greater Element I (LC 496)	Monotonic Stack	$O(n)$ time $O(n)$ space	- Use decreasing stack - Keep popping smaller elements - Map next greater for each
④	Daily Temperatures (LC 739)	Monotonic Stack	$O(n)$ time $O(n)$ space	- Store indices in stack - Pop while current > top temp - Answer = current_index - pop_index
⑤	Implement Queue Using Stacks (LC 232)	Two Stacks	Amortized $O(1)$ time $O(n)$ space	- For push → push in stack1 - For pop → move to stack2 if empty - Pop from stack2
⑥	Implement Stack Using Queues (LC 225)	Two Queues	$O(1)$ time (push) $O(n)$ time (pop) $O(n)$ space	- For push → enqueue and rotate - Latest element always at front - Pop is just dequeue
⑦	Evaluate Reverse Polish Notation (LC 150)	Stack	$O(n)$ time $O(n)$ space	- If number → push to stack - If operator → pop two, apply - Push result back
⑧	Largest Rectangle in Histogram (LC 84)	Monotonic Stack	$O(n)$ time $O(n)$ space	- Use increasing stack - Calculate area when popping - Add sentinel 0 at end



COMMON STACK / QUEUE PATTERNS

- | | | |
|------------------------------|-------------------------|----------------------------------|
| → Monotonic Increasing Stack | → Two Stack / Two Queue | → Next Greater / Smaller Element |
| → Monotonic Decreasing Stack | → Expression Evaluation | → Sliding Window Maximum |
| → Stack Simulation | → Parentheses Matching | → BFS (Using Queue) |



TIP:

Draw the stack / queue on paper. It helps to avoid corner cases and build correct intuition!



GOAL:

Solve problems with clean logic and focus on pattern recognition. Speed will come with practice!

MOST ASKED DSA INTERVIEW QUESTIONS

(0-3 YEARS) (5-20 LPA)

PAGE 7

7. BINARY SEARCH - MOST ASKED QUESTIONS



Binary Search reduces search space from $O(n)$ to $O(\log n)$.
Always think in terms of sorted data / monotonic condition.

#	QUESTION	PATTERN	EXPECTED COMPLEXITY	KEY IDEA / HINT
①	Binary Search (LC 704)	Binary Search	$O(\log n)$ $O(1)$ space	- Search in sorted array - Compare mid and move range
②	Find First and Last Position (LC 34)	Binary Search (Left & Right)	$O(\log n)$ $O(1)$ space	- Find leftmost and rightmost - Use modified binary search
③	Search in Rotated Sorted Array (LC 33)	Binary Search (Rotated Array)	$O(\log n)$ $O(1)$ space	- Find sorted half, then search - Handle pivot correctly
④	Find Peak Element (LC 162)	Binary Search (Peak)	$O(\log n)$ $O(1)$ space	- Compare mid with neighbors - Peak is greater than neighbors
⑤	Square Root of x (LC 69)	Binary Search (Answer Space)	$O(\log n)$ $O(1)$ space	- Search integer sqrt - Check $mid * mid \leq x$
⑥	Search in Answer Space (Binary Search on Range)	Binary Search (Answer Space)	$O(\log R)$ $O(1)$ space	- Binary search on range of answers - Useful in optimization problems



BINARY SEARCH TEMPLATE

```

left = start, right = end
while left <= right:
    mid = left + (right - left) // 2
    if condition(mid):
        possible answer, move left/right
    else:
        move the other direction
Return final answer
    
```

WHEN TO USE BINARY SEARCH

- Array is sorted
- You can define a monotonic condition
- When answer space is ordered (min/max possible)

COMMON TRICKS

- Left = mid + 1 (search right half)
- Right = mid - 1 (search left half)
- Be careful with conditions to avoid infinite loops
- Use long / 64-bit when dealing with large ranges

EDGE CASES

- Empty array
- Single element array
- All elements same
- Target less / greater than all elements

8. TREES & BST - MOST ASKED QUESTIONS



Trees are hierarchical structures. BST has extra property: left / root < right.

#	QUESTION	#	QUESTION
1	Tree Traversals (Pre, In, Post Order)	5	Check if Balanced Binary Tree
2	Maximum Depth of Binary Tree	6	Validate Binary Search Tree
3	Level Order Traversal	7	Lowest Common Ancestor (LCA)
4	Diameter of Binary Tree		

DFS vs BFS

- DFS → Go deep first (Stack / Recursion)
- BFS → Go level by level (Queue)
- Use DFS for depth related problems
- Use BFS for shortest path / level problems

BST PROPERTY

- Left subtree < Root < Right subtree
- Inorder traversal gives sorted order
- Insert / Search / Delete in $O(h)$ time
- h = height of tree ($O(\log n)$ for balanced BST)



Practice DSA daily, build patterns, solve problems consistently.
Small steps today, big results tomorrow! 🚀



MOST ASKED DSA INTERVIEW QUESTIONS

(0-3 YEARS) (5-20 LPA)

PAGE 7

7. BINARY SEARCH - MOST ASKED QUESTIONS



Binary Search reduces search space in half each step.
Know the template and variations well.

#	QUESTION	PATTERN	EXPECTED COMPLEXITY	KEY IDEA / HINT
①	Binary Search (LC 704)	Binary Search	$O(\log n)$	- Classic binary search template - Works on sorted array
②	Search First and Last Position (LC 34)	Binary Search (Modified)	$O(\log n)$	- Find left boundary and right boundary - Use two binary searches or lower_bound / upper_bound
③	Search in Rotated Sorted Array (LC 33)	Binary Search (Rotated Array)	$O(\log n)$	- Decide which half is sorted - Search in sorted half
④	Find Peak Element (LC 162)	Binary Search (1D Mountain)	$O(\log n)$	- Compare with neighbors - Move to the side with higher value
⑤	Square Root of X (LC 69)	Binary Search (Answer Space)	$O(\log n)$	- Search on answer space - Check $mid * mid \leq x$
⑥	Search in Sorted Matrix (LC 240)	Binary Search (2D Search)	$O(m + \log n)$	- Start from top-right or bottom-left - Eliminate row or column
⑦	Koko Eating Bananas (LC 875)	Binary Search (Answer Space)	$O(n \log \max)$	- Find minimum speed (k) - Check if possible in time h



BINARY SEARCH TEMPLATE

```

low = start, high = end
while low <= high:
    mid = low + (high - low) // 2
    if condition(mid) is True:
        high = mid - 1 # or high = mid (for first true)
    else:
        low = mid + 1 # or low = mid + 1 (for last true)
return best answer
    
```

TIPS

- Always check edge cases
- Think about monotonic property
- Use long long for large numbers
- Draw examples for tricky problems

COMMON BINARY SEARCH USE CASES

- ✓ Search in Sorted Array
- ✓ Answer Space Problems
- ✓ Optimization Problems
- ✓ Finding Boundaries (First/Last)



QUICK REMEMBER

- Binary Search works only on sorted data
- Time Complexity is always $O(\log n)$
- Think "Is the answer possible?" for answer space problems



KEY TAKEAWAY

- Binary Search is a powerful pattern
- Learn variations and apply everywhere
- Practice edge cases and boundaries



Master the patterns, practice regularly,
and binary search will become second nature!



MOST ASKED DSA INTERVIEW QUESTIONS

(0-3 YEARS) (5-20 LPA)

PAGE 8

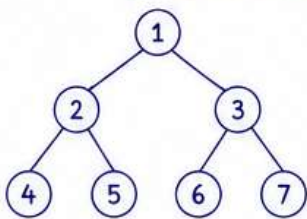
8. TREES & BST - MOST ASKED QUESTIONS (Detailed)



Trees are hierarchical. BST follows: left < root < right.
Master traversals + common tree patterns.

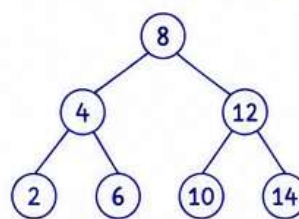
#	QUESTION	PATTERN / CONCEPT	EXPECTED COMPLEXITY	KEY IDEA / HINT
①	Tree Traversals (Pre, In, Post Order) (LC 144, 145, 94)	DFS (Recursion / Stack)	$O(n)$ time $O(h)$ space	- Pre: Root → Left → Right - In: Left → Root → Right - Post: Left → Right → Root
②	Maximum Depth of Binary Tree (LC 104)	DFS (Recursion)	$O(n)$ time $O(h)$ space	- Height = 1 + max(left, right) - Base: null → 0
③	Level Order Traversal (LC 102)	BFS (Queue)	$O(n)$ time $O(w)$ space (w = max width)	- Use queue - Push children level by level - Track level size for each level
④	Diameter of Binary Tree (LC 543)	DFS (Post Order)	$O(n)$ time $O(h)$ space	- Diameter = longest path - Pass heights up from children - Update global max
⑤	Check if Balanced Binary Tree (LC 110)	DFS (Bottom Up)	$O(n)$ time $O(h)$ space	- Height diff of left & right should be ≤ 1 - If any subtree not balanced, return -1
⑥	Validate Binary Search Tree (LC 98)	DFS (Inorder)	$O(n)$ time $O(h)$ space	- Inorder of BST is sorted - Use prev pointer to check current > prev
⑦	Lowest Common Ancestor (LCA) (LC 236)	DFS	$O(n)$ time $O(h)$ space	- If root is null → return null - If root == p or q → return root - If left & right not null → root

TREE TRAVERSAL VISUAL



Preorder (Root, Left, Right)
→ 1, 2, 4, 5, 3, 6, 7
Inorder (Left, Root, Right)
→ 4, 2, 5, 1, 6, 3, 7
Postorder (Left, Right, Root)
→ 4, 5, 2, 6, 7, 3, 1
Level Order (BFS)
→ 1, 2, 3, 4, 5, 6, 7

BST PROPERTY



For any node:
• Left < Node
• Right > Node

Inorder of BST
always gives
sorted order.

DFS (Depth First Search)

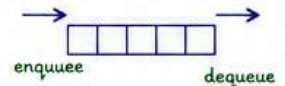
- Go deep first
- Use Stack / Recursion
- Good for path, components, backtracking, LCA



DFS vs BFS

BFS (Breadth First Search)

- Explore level by level
- Use Queue
- Good for shortest path, level order, minimum steps problems



TIME & SPACE COMPLEXITY CHEATSHEET

- n = number of nodes
- h = height of tree (log n for balanced, n for skewed)
- DFS (Recursion): Time $O(n)$, Space $O(h)$
- BFS (Queue): Time $O(n)$, Space $O(w)$
where w = max width of tree

COMMON TREE PATTERNS TO RECOGNIZE

- ✓ Traverse everything → DFS / BFS
- ✓ Path / Root to Leaf → DFS
- ✓ Level wise / Shortest → BFS
- ✓ Height / Depth / Balanced → DFS (Bottom Up)
- ✓ BST operations / Validation → Inorder / Range



Understand the pattern, not just the code.
Trees look scary at first, then become your strength! 🚀



MOST ASKED DSA INTERVIEW QUESTIONS

(0-3 YEARS) (5-20 LPA)

PAGE 9

9. HEAPS (PRIORITY QUEUE) - MOST ASKED QUESTIONS



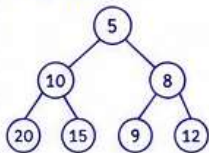
Heap is a complete binary tree that satisfies the heap property.
Used to implement Priority Queue efficiently.

#	QUESTION	PATTERN / CONCEPT	EXPECTED COMPLEXITY	KEY IDEA / HINT
①	Kth Largest Element in Array (LC 215)	Heap (Min Heap)	$O(n \log k)$ time $O(k)$ space	- Use Min Heap of size k - Push first k elements - For remaining, replace if greater - Top gives kth largest
②	K Closest Points to Origin (LC 973)	Heap (Max Heap)	$O(n \log k)$ time $O(k)$ space	- Use Max Heap of size k - Push first k points by distance - Replace if new point is closer
③	Merge K Sorted Lists (LC 23)	Heap (Min Heap)	$O(N \log k)$ time $O(k)$ space (N = total nodes)	- Push head of all lists in Min Heap - Pop min, push next of that list - Repeat until heap empty
④	Find Median from Data Stream (LC 295)	Two Heaps (Max + Min)	$O(\log n)$ time $O(n)$ space	- Max Heap for smaller half - Min Heap for larger half - Balance sizes, get median
⑤	Top K Frequent Elements (LC 347)	Heap (Min Heap)	$O(n \log k)$ time $O(k)$ space	- Count frequency using Map - Push (freq, element) in Min Heap - Keep size = k
⑥	Sort Nearly Sorted Array (LC 1046)	Heap (Min Heap)	$O(n \log k)$ time $O(k)$ space	- Elements are at most k away - Use Min Heap of size $k + 1$ - Pop min and push next

HEAP TYPES

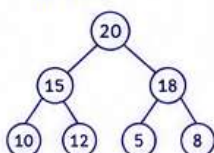
MIN HEAP

- Parent $\leq$ Child
- Smallest element at root



MAX HEAP

- Parent $\geq$ Child
- Largest element at root



HEAP OPERATIONS COMPLEXITY

- Insert (push) $O(\log n)$
- Delete Top (pop) $O(\log n)$
- Peek / Top $O(1)$
- Build Heap $O(n)$
- Heapify $O(\log n)$

WHEN TO USE HEAP?

- K largest / smallest elements
- Merge K sorted lists
- Priority based processing
- Streaming median
- Scheduling / Simulation
- Dijkstra's Algorithm
- Huffman Coding

HEAP IMPLEMENTATION (ARRAY REPRESENTATION)

For node at index i (0-indexed):

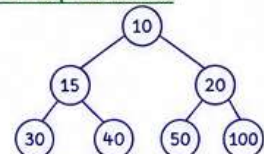
Left Child = $2i + 1$

Right Child = $2i + 2$

Parent = $(i - 1) / 2$

Index:	0	1	2	3	4	5	6
Value:	10	15	20	30	40	50	100

Tree Representation:

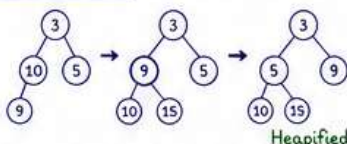


HEAPIFY (MIN HEAP) - DOWN HEAP

Compare node with its children.

Swap with smallest child.

Repeat until heap property holds.



QUICK TIPS

- ✓ Use Min Heap for "Top K Largest" problems.
- ✓ Use Max Heap for "Top K Smallest" problems.
- ✓ Always keep heap size limited for optimization.
- ✓ Convert problem to "Maintain Top K" pattern.



Understand the data structure, then code becomes easy.
Heaps make priority work effortless! 🚀



MOST ASKED DSA INTERVIEW QUESTIONS

(0-3 YEARS) (5-20 LPA)

PAGE 10

10. GRAPHS - MOST ASKED QUESTIONS



Graphs are used to represent relationships between entities.

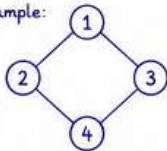
Types: Directed / Undirected, Weighted / Unweighted

#	QUESTION	PATTERN / CONCEPT	EXPECTED COMPLEXITY	KEY IDEA / HINT
①	Number of Islands (LC 200)	DFS / BFS (Grid Graph)	$O(m \times n)$ time $O(m \times n)$ space	- Explore all 4 directions - Mark visited cells
②	Clone Graph (LC 133)	DFS / BFS (Graph Traversal)	$O(V + E)$ time $O(V)$ space	- Use HashMap to store cloned nodes - Avoid revisiting nodes
③	Detect Cycle in Undirected Graph (LC 547)	DFS / Union Find	$O(V + E)$ time $O(V)$ space	- DFS: check parent node - Union Find: cycle in union
④	Course Schedule (LC 207)	BFS (Topological Sort - Kahn's Algo)	$O(V + E)$ time $O(V)$ space	- Build in-degree array - If all nodes processed $\rightarrow$ possible, else not
⑤	Minimum Spanning Tree (Prim's / Kruskal's)	Greedy Algorithm	$O(E \log V)$ (using PQ)	MST connects all nodes with minimum total weight
⑥	Dijkstra's Algorithm (Shortest Path)	Greedy + PQ	$O((V + E) \log V)$	- Use Min Heap - Works for non-negative weights only
⑦	Topological Sort (LC 210)	BFS / DFS	$O(V + E)$ time $O(V)$ space	- For DAG (Directed Acyclic Graph) - Order of courses / tasks

GRAPH REPRESENTATION

- Adjacency List: Space Efficient
- Adjacency Matrix: Faster lookups

Example:



Adjacency List

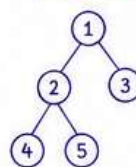
1 $\rightarrow$ [2, 3]
2 $\rightarrow$ [1, 4]
3 $\rightarrow$ [1, 4]
4 $\rightarrow$ [2, 3]

Adjacency Matrix

	1	2	3	4
1	0	1	1	0
2	1	0	0	1
3	1	0	0	1
4	0	1	1	0

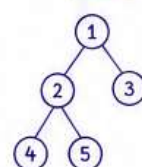
COMMON GRAPH TRAVERSALS

BFS (Level Order)



Use Queue
Visit Level by Level

DFS (Depth First)



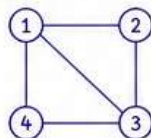
Use Stack / Recursion
Go as deep as possible

CYCLE DETECTION (UNDIRECTED GRAPH)

DFS Approach:

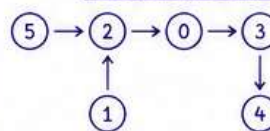
- If visited adjacent node is not parent $\rightarrow$ cycle exists.
- Keep track of parent while DFS.

Example:



Cycle exists because from 1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 4, node 1 is visited again (not parent).

TOPOLOGICAL SORT (KAHN'S ALGORITHM)



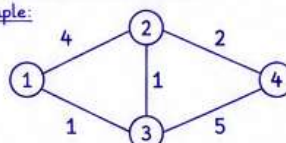
- Calculate in-degree of all nodes.
- Push all nodes with in-degree 0 into queue.
- Pop from queue, add to result.
- Decrease in-degree of neighbors.
- Repeat until queue is empty.

DIJKSTRA'S ALGORITHM (SHORTEST PATH)

Steps:

- Initialize distance of all nodes as ∞ , source = 0.
- Use Min Heap to pick node with minimum distance.
- Update distances of its neighbors.
- Repeat until all nodes are processed.

Example:



Shortest distances from node 1:

- 1 $\rightarrow$ 0
- 2 $\rightarrow$ 2 (1 $\rightarrow$ 3 $\rightarrow$ 2)
- 3 $\rightarrow$ 1 (1 $\rightarrow$ 3)
- 4 $\rightarrow$ 4 (1 $\rightarrow$ 3 $\rightarrow$ 2 $\rightarrow$ 4)

COMMON TIPS FOR GRAPH PROBLEMS

- Identify graph type (Directed / Undirected).
- Choose correct traversal (BFS / DFS).
- Consider edge weights (use Dijkstra / Prim / Kruskal).
- Check for cycles when required.
- Draw the graph - it helps a lot!



TIME & SPACE COMPLEXITY CHEATSHEET

- V = Number of Vertices (Nodes)
- E = Number of Edges
- DFS / BFS : $O(V + E)$
- Dijkstra : $O((V + E) \log V)$
- MST (Prim/Kruskal) : $O(E \log V)$
- Topological Sort : $O(V + E)$



Master Graphs, master relationships!
Great for system design, routing, dependencies & more! 🚀



MOST ASKED DSA INTERVIEW QUESTIONS

(0-3 YEARS) (5-20 LPA)

PAGE 11

11. DYNAMIC PROGRAMMING (DP) - MOST ASKED QUESTIONS



DP is all about solving problems by breaking them into overlapping subproblems.

Steps: 1) Define state 2) Write recurrence 3) Choose DP approach (Top-Down / Bottom-Up)

#	QUESTION	PATTERN / CONCEPT	EXPECTED COMPLEXITY	KEY IDEA / HINT
①	Climbing Stairs (LC 70)	1D DP (Fibonacci)	$O(n)$ time $O(1) / O(n)$ space	$dp[i]$ = ways to reach step i $dp[i] = dp[i-1] + dp[i-2]$
②	House Robber (LC 198)	1D DP	$O(n)$ time $O(1)$ space	- At each step, choose / skip $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$
③	Longest Increasing Subsequence (LC 300)	1D DP / Binary Search	$O(n^2)$ time $O(n)$ space	$dp[i]$ = length of LIS ending at i - Optimize to $O(n \log n)$
④	Edit Distance (LC 72)	2D DP (Strings)	$O(m \times n)$ time $O(m \times n)$ space	- Insert / Delete / Replace $dp[i][j]$ = min of 3 operations
⑤	0/1 Knapsack (LC 416)	2D / 1D DP	$O(n \times W)$ time $O(W)$ space	$dp[i][w]$ = max value using first i items & capacity w
⑥	Coin Change 2 (LC 518)	2D DP	$O(n \times \text{amount})$ time $O(\text{amount})$ space	- Count ways to make amount - Order doesn't matter
⑦	Longest Common Subsequence (LC 1143)	2D DP (Strings)	$O(m \times n)$ time $O(m \times n)$ space	- If chars match $\rightarrow +1$ - Else $\rightarrow \max$ of top / left
⑧	Partition Equal Subset Sum (LC 416)	Subset Sum DP	$O(n \times \text{sum})$ time $O(\text{sum})$ space	- Check if $\text{sum}/2$ is possible - Boolean DP (true/false)

DP APPROACHES

1. Top-Down (Memoization)

- Use recursion + memo array
- Easier to write & understand
- Good for sparse states

2. Bottom-Up (Tabulation)

- Build from base cases
- Usually faster (no recursion)
- Preferred in interviews

COMMON DP PATTERNS

- 1D DP: Linear sequences, choices
- 2D DP: Grids, strings, intervals
- Knapsack Pattern: Maximize value
- Subsequence Pattern: LIS, LCS
- Partition Pattern: Subset Sum, Equal Partition
- DP on Trees: Tree DP, Path DP

EXAMPLE 1: CLIMBING STAIRS (DP)

Problem: Given n stairs, you can climb 1 or 2 steps at a time. Find number of ways to reach n th stair.

Recurrence: $dp[i] = dp[i-1] + dp[i-2]$

Base: $dp[0] = 1, dp[1] = 1$

Answer: $dp[n]$

Bottom-Up (Tabulation)

$dp[0] = 1, dp[1] = 1$

for i from 2 to n :

$dp[i] = dp[i-1] + dp[i-2]$

return $dp[n]$

Dry Run ($n = 5$)

i	0	1	2	3	4	5
dp[i]	1	1	2	3	5	8

Answer = 8

EXAMPLE 2: EDIT DISTANCE (DP)

Problem: Minimum operations to convert word1 to word2. (Insert / Delete / Replace)

State: $dp[i][j]$ = min operations to convert first i chars of word1 to first j chars of word2

Base:
 $dp[i][0] = i$ (delete all i chars)
 $dp[0][j] = j$ (insert all j chars)

Transition:

if $word1[i-1] == word2[j-1]$:

$dp[i][j] = dp[i-1][j-1]$

else:

$dp[i][j] = 1 + \min$ (

$dp[i-1][j],$ # delete

$dp[i][j-1],$ # insert

$dp[i-1][j-1]$ # replace

Example: word1 = "horse"
word2 = "ros"

		h	r	o	s
-	0	1	2	3	
h	1	1	2	3	
o	2	2	1	2	
r	3	2	2	2	
s	4	3	3	2	
e	5	4	4	3	

Answer = 3

SPACE OPTIMIZATION TIPS

- Many DP problems can be done in $O(1)$ or $O(n)$ space using rolling arrays.
- Only keep current & previous state.

prev $\rightarrow$ [1] [2] [3] [5] [8]
curr $\rightarrow$ [] [] [] [] [] Reuse & update!

WHEN TO USE DP?

- ✓ Overlapping Subproblems
- ✓ Optimal Substructure
- ✓ Choice at each step
- ✓ Can be broken into states

DP TEMPLATE (2D Example)

$dp = [[0] * (n+1) \text{ for } _ \text{ in range}(m+1)]$
for i in range($m+1$):
for j in range($n+1$):
if $i == 0$ or $j == 0$:
 $dp[i][j] = \text{base_case}$
else:
 $dp[i][j] = \text{transition}$



DP turns complex problems into simple steps!
Practice patterns, master states, crack interviews!



MOST ASKED DSA INTERVIEW QUESTIONS

(0-3 YEARS) (5-20 LPA)

PAGE 12

12. GRAPH ALGORITHMS – MOST ASKED QUESTIONS



Graph algorithms help us find shortest path, minimum cost, connectivity, components etc. Know when to use which!

#	QUESTION	ALGORITHM	EXPECTED COMPLEXITY	KEY IDEA / HINT
①	Shortest Path in Unweighted Graph (LC 1091)	BFS	$O(V + E)$ time $O(V)$ space	- BFS from source - First time we visit = shortest
②	Shortest Path in Weighted Graph (LC 743)	Dijkstra's Algorithm (Min Heap)	$O((V + E) \log V)$ time $O(V)$ space	- Non-negative weights - Greedy: always pick minimum distance node
③	Detect Negative Cycle (LC 209)	Bellman-Ford Algorithm	$O(V \times E)$ time $O(V)$ space	- Relax all edges $V-1$ times - One more relaxation that improves $\rightarrow$ cycle
④	Find All Bridges in Graph (LC 1192)	Tarjan's Algorithm (DFS + Low Time)	$O(V + E)$ time $O(V)$ space	An edge (u,v) is bridge if $low[v] > disc[u]$
⑤	Find Articulation Points (LC 1125)	Tarjan's Algorithm (DFS + Low Time)	$O(V + E)$ time $O(V)$ space	Vertex whose removal increases no. of connected components
⑥	Minimum Spanning Tree (MST) (LC 1584)	Kruskal's / Prim's Algorithm	$O(E \log E)$ or $O((V + E) \log V)$	MST connects all nodes with minimum total weight
⑦	Check if Graph is Bipartite (LC 785)	BFS / DFS (Coloring)	$O(V + E)$ time $O(V)$ space	- Try to color using 2 colors - If conflict $\rightarrow$ not bipartite

SHORTEST PATH – QUICK GUIDE

- Unweighted Graph $\rightarrow$ BFS
- Weighted (non-negative) $\rightarrow$ Dijkstra
- Negative weights $\rightarrow$ Bellman-Ford
- Need only yes/no reachability $\rightarrow$ DFS/BFS

DIJKSTRA (MIN HEAP) – IDEA

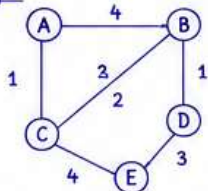
- Keep distance[] array
- Min Heap stores (dist, node)
- Pop node with smallest dist
- Relax all its neighbours

BELLMAN-FORD – IDEA

- Relax all edges $V-1$ times
- If any edge can still be relaxed on V th time $\rightarrow$ Negative Cycle
- Works with negative weights

DIJKSTRA EXAMPLE

Graph:



Source = A
Steps (dist[]):
Init: A=0, B=∞, C=∞, D=∞, E=∞
Pick A(0) $\rightarrow$ relax: B=4, C=1
Pick C(1) $\rightarrow$ relax: B=3, E=5
Pick B(3) $\rightarrow$ relax: D=4
Pick D(4) $\rightarrow$ relax: E=5 (unchanged)
Pick E(5) $\rightarrow$ done
Final dist: A=0, B=3, C=1, D=4, E=5

KRUSKAL vs PRIM

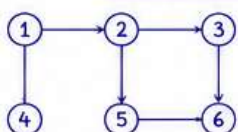
Kruskal (Edge based)

- Sort all edges
- Add smallest edge
- If it doesn't form cycle
- Use Disjoint Set (DSU)

Prim (Node based)

- Start from any node
- Grow MST by adding min edge to new node
- Use Min Heap

TARJAN – BRIDGE FINDING

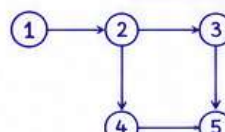


Edge (u,v) is a bridge if:

$$low[v] > disc[u]$$

Here, edges (1,4) and (3,6) are bridges.

TARJAN – ARTICULATION POINTS



Node u is articulation point if:

$$low[v] \geq disc[u]$$

(v is child of u in DFS tree)
Here, nodes 2 and 3 are articulation points.

GRAPH ALGORITHMS CHEATSHEET

- BFS / DFS $\rightarrow O(V + E)$
- Dijkstra (Min Heap) $\rightarrow O((V + E) \log V)$
- Bellman-Ford $\rightarrow O(V \times E)$
- Floyd-Warshall $\rightarrow O(V^3)$
- Kruskal (MST) $\rightarrow O(E \log E)$
- Prim (MST) $\rightarrow O((V + E) \log V)$

Use Which?

- Shortest path (unweighted) $\rightarrow$ BFS
- Shortest path (non-negative) $\rightarrow$ Dijkstra
- Negative weights / cycle $\rightarrow$ Bellman-Ford
- All pairs shortest path $\rightarrow$ Floyd-Warshall
- Minimum spanning tree $\rightarrow$ Kruskal / Prim

Remember!

- BFS = level by level
- DFS = go deep
- Greedy is your friend 😊
- Draw the graph, it helps!



Master graph algorithms, master problems!
They are the heart of DSA. 🚀



MOST ASKED DSA INTERVIEW QUESTIONS

(0-3 YEARS) (5-20 LPA)

PAGE 13

13. GREEDY ALGORITHMS - MOST ASKED QUESTIONS



Greedy picks the best option at each step with the hope of global optimum.

Works when problem has Greedy Choice Property + Optimal Substructure.

#	QUESTION	ALGORITHM	EXPECTED COMPLEXITY	KEY IDEA / HINT
①	Activity Selection (LC 435)	Greedy (Sort by End Time)	$O(n \log n)$ time $O(1)$ space	- Sort activities by end time - Pick current, then next with start $\geq$ last end
②	Fractional Knapsack (LC 1029)	Greedy (Sort by Value/Weight)	$O(n \log n)$ time $O(1)$ space	- Sort items by value/weight ratio desc - Take fraction if full item doesn't fit
③	Minimum Number of Coins (Infinite Supply) (LC 322)	Greedy (Sort Coins Desc)	$O(n + \text{amount})$ time $O(1)$ space	- Pick largest coin $\leq$ amount - Repeat for remaining
④	Jump Game II (LC 45)	Greedy (Level by Level)	$O(n)$ time $O(1)$ space	- Expand farthest reachable range at each jump - Count jumps
⑤	Merge Intervals (LC 56)	Greedy (Sort by Start)	$O(n \log n)$ time $O(n)$ space	- Sort by start - Merge if overlapping
⑥	Minimum Cost to Connect Ropes (LC 1167)	Greedy + Min Heap	$O(n \log n)$ time $O(n)$ space	- Always connect two smallest ropes first - Push sum back to heap
⑦	Gas Station (LC 134)	Greedy	$O(n)$ time $O(1)$ space	- If total gas $<$ total cost $\rightarrow -1$ - Else, find valid start point

1. ACTIVITY SELECTION EXAMPLE

Activities (start, end):

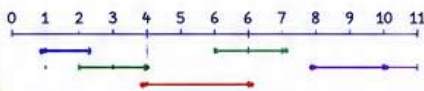
(1,4) (3,5) (0,6) (5,7) (3,9) (5,9) (6,10)

Sorted by end time:

(1,4) (3,5) (0,6) (5,7) (6,10) (3,9) (5,9) (8,11)

Pick: (1,4) $\rightarrow$ (5,7) $\rightarrow$ (8,11)

Max activities = 3



2. FRACTIONAL KNAPSACK EXAMPLE

Capacity = 50

Item	Wt	Val	Val/Wt	Sorted by Val/Wt:
1	10	60	6.0	1 (6.0)
2	20	100	5.0	2 (5.0)
3	30	120	4.0	3 (4.0)

Take item 1 (10) $\rightarrow$ val = 60

Take item 2 (20) $\rightarrow$ val = 100 (total wt = 30)

Take 20/30 of item 3 $\rightarrow$ val = 20/30 * 120 = 80

Total Value = 60 + 100 + 80 = 240

4. JUMP GAME II EXAMPLE

nums = [2,3,1,1,4]

Index: 0 1 2 3 4

Value: [2, 3, 1, 1, 4]

Range Expansion:

Jump 1: from 0 $\rightarrow$ can reach [1,2]

Jump 2: from [1,2] $\rightarrow$ can reach [3,4]

Jump 3: from [3,4] $\rightarrow$ reach end

Min Jumps = 2

5. MERGE INTERVALS EXAMPLE

Intervals: [(1,3), (2,6), (8,10), (15,18)]

Sorted by Start: [(1,3), (2,6), (8,10), (15,18)]

Merge:

[1,3] [2,6] $\rightarrow$ [1,6] $\rightarrow$ [8,10]

[8,10] [15,18] $\rightarrow$ [15,18]

Result: [(1,6), (8,10), (15,18)]

6. CONNECT ROPES EXAMPLE

Ropes: [4, 3, 2, 6]

Min Heap:

Step 1: take 2 + 3 = 5 (cost = 5)

Heap $\rightarrow$ [4, 5, 6]

Step 2: take 4 + 5 = 9 (cost = 14)

Heap $\rightarrow$ [6, 9]

Step 3: take 6 + 9 = 15 (cost = 29)

Total Minimum Cost = 29

7. GAS STATION EXAMPLE

gas = [1, 2, 3, 4, 5]

cost = [3, 4, 5, 1, 2]

Index	0	1	3	4
Diff	-2	-2	3	3

Total = 0 $\rightarrow$ Possible

Start Index = 3

GREEDY CHOICE PROPERTY

A problem has greedy choice property if we can make the locally optimal choice and still get global optimum.

OPTIMAL SUBSTRUCTURE

Optimal solution can be built from optimal solutions of subproblems.



WHEN TO USE GREEDY?

- Activity / Interval Scheduling
- Minimum / Maximum Problems
- Problems with Sorting + Selection
- Resource allocation
- Huffman Coding
- MST (Kruskal / Prim)
- Dijkstra (non-negative weights)

GREEDY TIPS

- ✓ Prove correctness (if possible)
 - ✓ Sort smartly
 - ✓ Pick best & move forward
 - ✓ Avoid backtracking
 - ✓ Sometimes not always works!
- Ex: 0/1 Knapsack (not greedy)

QUICK CHECKLIST

- Can I make local best choice?
- Does it stay optimal overall?
- Can I solve subproblems independently?
- If yes $\rightarrow$ Try Greedy!



Pick smart at every step,
Greedy makes it look easy! **100**

